

CAPSTONE PROJECT

USECASE 3 – Statistical Insights & Pattern Detection

Name : Mamidoju.Vishal

Email : vishalmamidoju5@gmail.com

Objective: Use numerical and visual analysis to derive deeper insights.

Tasks

1.Crime Intensity by Time:

- Use Pandas groupby to get hourly distribution
- `df['Hour'] = df['Date'].dt.hour`
- `crimes_by_hour = df.groupby('Hour').size()`
- Line plot of crimes per hour of day.

Code:

```
# ===== 1. CRIME INTENSITY BY TIME =====

df["crime_hour"] = df["date"].dt.hour
hourly_crimes = df.groupby("crime_hour").size()

print("\n===== CRIME FREQUENCY BY HOUR =====")
print(hourly_crimes)

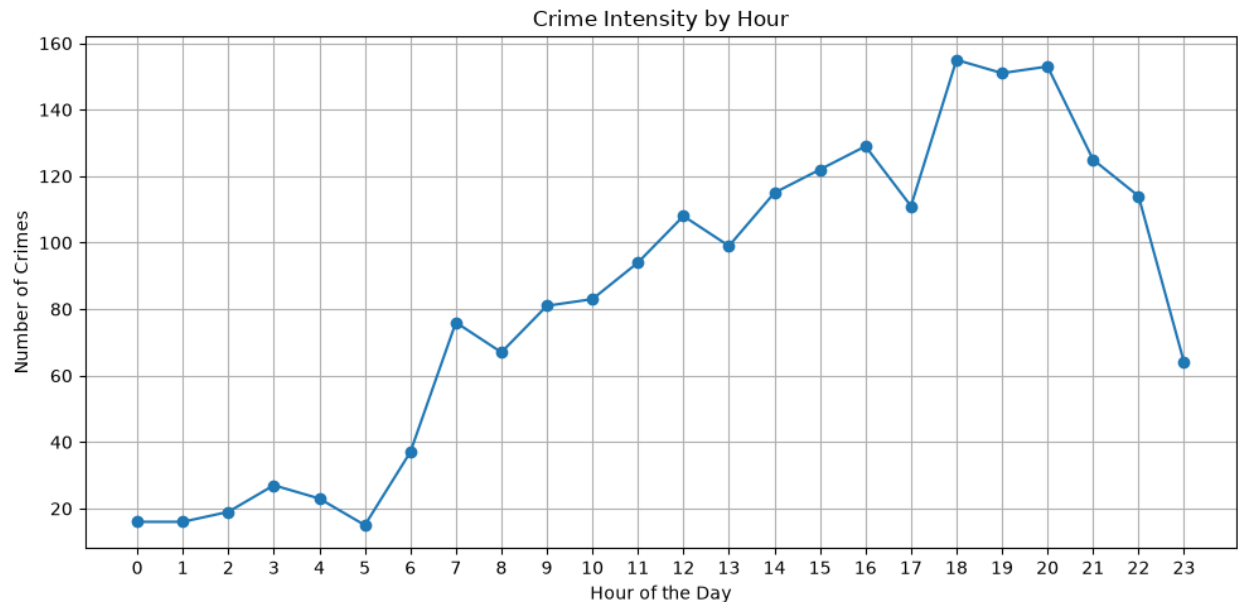
peak_hour = hourly_crimes.idxmax()
peak_crimes = hourly_crimes.max()

print("\n===== PEAK CRIME TIME =====")
print("Peak Crime Hour:", peak_hour)
print("Number of Crimes:", peak_crimes)

# ===== GRAPH 1: CRIME BY HOUR =====

plt.figure(figsize=(10, 5))
plt.plot(hourly_crimes.index, hourly_crimes.values, marker="o")
plt.title("Crime Intensity by Hour")
plt.xlabel("Hour of the Day")
plt.ylabel("Number of Crimes")
plt.xticks(range(0, 24))
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(CHART_DIR, "crime_by_hour.png"), bbox_inches="tight")
plt.close()
```

Output:



2.Community Area Clusters Using NumPy:

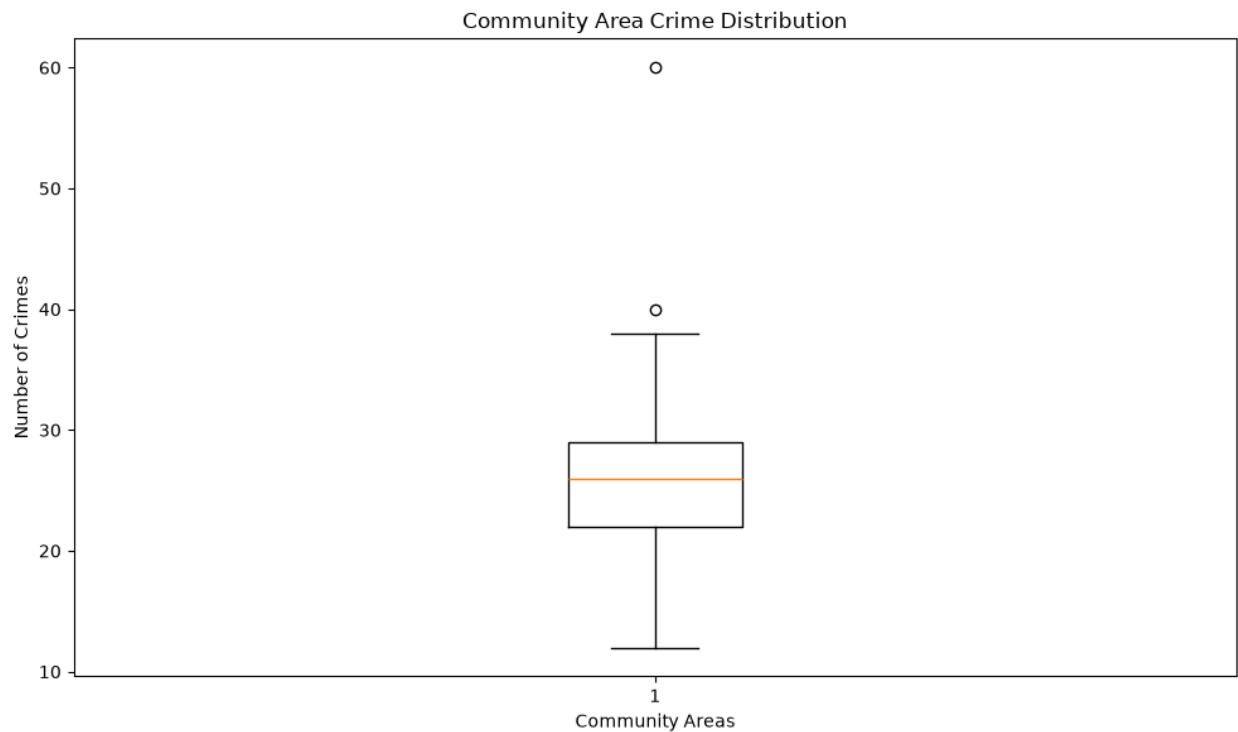
- Use NumPy functions to compute mean crime per community area.
- Identify which areas are extreme outliers in terms of crime counts (Use box plot to identify the outliers and their...)

Code:

```
top_10_communities = community_stats.head(10)

plt.figure(figsize=(10, 6))
plt.bar(
    top_10_communities["community_code"].astype(str),
    top_10_communities["crime_count"]
)
plt.title("Top 10 Community Areas by Crime Count")
plt.xlabel("Community Code")
plt.ylabel("Number of Crimes")
plt.tight_layout()
plt.savefig(os.path.join(CHART_DIR, "community_crimes.png"), bbox_inches="tight")
plt.close()
```

Output:



3.Crime Cross-Correlation:

- Use Pandas .corr() on numeric features (Year, Month, Arrest, etc.)
- Visualize correlation matrix with a heatmap.

Code:

```
print("\n===== CRIME CROSS-CORRELATION =====")

numeric_data = df.select_dtypes(include=[np.number])
correlation_matrix = numeric_data.corr()

print(correlation_matrix)
```


Insight:

This use case analyzes Chicago crime data using **time-based analysis, NumPy statistical analysis, and correlation analysis**.

- **Crime Intensity by Time:** The data is grouped by hour to identify the time when crime occurs most frequently. A line graph is generated to visualize hourly crime patterns and determine the peak crime hour.
- **Community Area Analysis:** NumPy is used to calculate the number of crimes in each community area, along with the **average, maximum, minimum, and standard deviation** of crime counts. The top 10 community areas with the highest crime counts are displayed using a bar chart.
- **Crime Cross-Correlation:** Numerical columns are analyzed using a correlation matrix to identify relationships between different crime-related variables. The correlation matrix is visualized as a heatmap-like chart.